RECON

Who am I?

Co-founder of Recon

LSR at Spearbit / C4

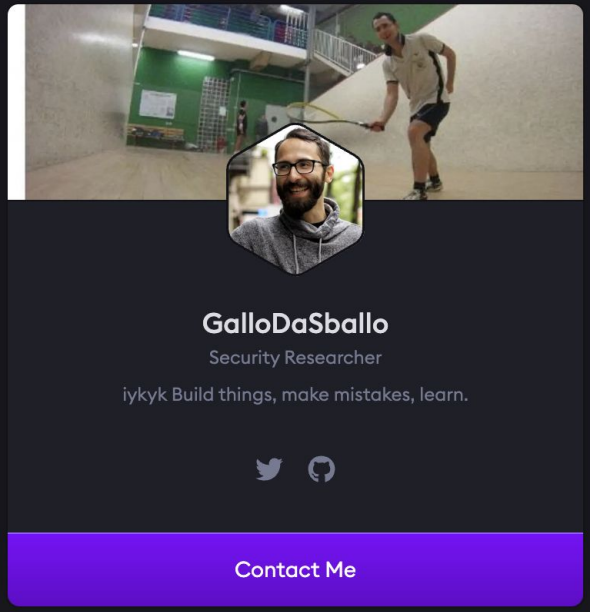
SC Dev for Badger, Hundreds of Millions TVL

$500k in awards

$20 MLN in mainnet exploits prevented in 2025

Returning Customers | $3 BLN TVL

Centrifuge Badger corn



What do we offer?

- Audit powered by Invariant Testing
- Cloud Fuzzing + AI Automations

What’s pretty obvious as a form of testing to engineers?

Assertions about storage values

Deployment script

Our experience can offer improvements, but not a 0 to 1.

These are not blockers.

What’s a new skill that has zero value?

Getting to coverage

Has 0 value in it of itself, it’s a means to an end

But due to how difficult it is, we need to learn skills tied to it.

The easier we make it to get to coverage, the more likely it is that people will use this technology.

What are the biggest issues for normal users?

- Domain specific knowledge (handlers, clamping, coverage, workflow)

- False Sense of Security.

- Maintain this (massive) amount of work.
- Each update to the code tends to require work to ensure the tests are meaningful again. This is extra work that is only needed with fuzzing.

What are the biggest hurdles for fuzzing today?

- Dictionaries usage is unsophisticated
- Sequence Length / Effort algorithms lead the fuzzer to be much less efficient than it could be
- Reentrancy fundamentally hard to explore.
- Bytes encoding (Lack of interface to generate sequences)
- Multicall (Stack type sequences with very similar state space)

If we had to do none of this, then we'd all be using this technology.

Dictionary Limitations

Morpho famously having issues with reaching coverage without hardcoding the market id is an indicator that there's massive room for quick improvement.

Similar issues apply to more nuanced codebase, e.g. Liquity Batch ID.

A lack of dictionary sophistication forces us to use Dynamic Clamping as a means to compensate for the dictionary.

Struct that represents marketId

```
function morpho_liquidate(MarketParams memory marketParams, address borrower, uint256
    morpho.liquidate(marketParams, borrower, seizedAssets, repaidShares, data);
}

function morpho_repay(MarketParams memory marketParams, uint256 assets, uint256 shares
    morpho.repay(marketParams, assets, shares, onBehalf, data);
}

function morpho_setAuthorization(address authorized, bool newIsAuthorized) public asAc
    morpho.setAuthorization(authorized, newIsAuthorized);
}

function morpho_setAuthorizationWithSig(Authorization memory authorization, Signature
    morpho.setAuthorizationWithSig(authorization, signature);
}

function morpho_setFee(MarketParams memory marketParams, uint256 newFee) public asActo
    morpho.setFee(marketParams, newFee);
}
```

Sequence Length / Effort

Some functions reach 100% SP exploration in just a few call.

Some functions can never be reached.

Others require different level of effort.

E.g. Withdrawals by definition require a deposit of some sort.

Live coverage as well as some form of whitebox static analysis (e.g. storage dependency analysis) should ensure the fuzzer keeps “digging” where it matters.

Reentrancy fundamentally unsolved

Reentrant Handler

Snapshot (ityFuzz) into new coverage metrics could help handle this

Currently:

- Recommend flagging reentrancy but don’t expect the fuzzer to be able to generate an exploit

-> Show architecture

Reentrancy Realistic

UnitTest stub | dependencies | uml | funcSigs | draw.io

```
contract ReentrantHandlerQueue {

    struct Call {
        address target;
        bytes data;
        uint256 value;
    }

    Call[] public calls;

    ftrace | funcSig
    function addCall(address target, bytes memory data, uint256 value) public {
        calls.push(Call(target, data, value));
    }

    ftrace
    fallback() external payable {
        for (uint256 i = 0; i < calls.length; i++) {
            (bool success, bytes memory result) = calls[i].target.call{value: calls[i].value}(calls[i].data);
            if (!success) {
                revert(string(result));
            }
        }

        calls = new Call[](0);
    }
}
```

Reentrancy Ideal

```
UnitTest stub | dependencies | uml | funcSigs | draw.io
contract ReentrantHandlerIdeal {

    // TargetFunctions for each option
    ftrace | funcSig | ▷ Debug
    function option1() public {}
    ftrace | funcSig | ▷ Debug
    function option2() public {}

    // Reentrnacy handler
    // Tells the fuzzer to treat this contract as the target contract
    ftrace
    fallback() external payable {
        vm.fuzzFormHere();
    }
}
```

Bytes Encoding

Bytes are fundamentally randomized + use the dictionary

As humans, we can fairly easily determine other forms of relations for those bytes. However, we lack a easy way to do that.

Fuzzing bytes may be permanently require a separate framework due to how specific it is. Although it’s not fundamentally a per-protocol issue. But it indicates why libAFL can be a better approach for some codebases.

Bytes Consumer example - 1

```
enum Action {
    CALL,
    DELEGATECALL,
    STATICCALL,
    REVERT,
    RETURN
}
```

```
struct StaticCallPayload {
    address target;
    bytes data;
    uint256 value;
    uint256 saveToSlot;
}

struct RevertPayload {
    uint256 readFromSlot;
}
```

Bytes Consumer example - 2

```
ftrace | funcSig
function consume(Action action, bytes memory data) public {
    if (action == Action.CALL) {
        _callAndSaveToSlot(Action.CALL, data);
    } else if (action == Action.DELEGATECALL) {
        _callAndSaveToSlot(Action.DELEGATECALL, data);
    } else if (action == Action.STATICCALL) {
        _callAndSaveToSlot(Action.STATICCALL, data);
    } else if (action == Action.REVERT) {
        RevertPayload memory revertPayload = abi.decode(data, (RevertPayload));
        revert(string(abi.encodePacked(revertPayload.readFromSlot)));
    } else if (action == Action.RETURN) {
        ReturnPayload memory returnPayload = abi.decode(data, (ReturnPayload));
        uint256 slot = returnPayload.readFromSlot;
        uint256 value;
        assembly {
            value := sload(slot)
        }

        assembly {
            return(value, 32)
        }
    }
}
```

How do you fuzz it?

```
// Hardcoded one step and check
ftrace | funcSig
function callAndCheck(BytesConsumer.Action action↑, bytes memory data↑) public {
    bytes memory result = bytesConsumer.consume(action↑, data↑);
    bytes memory expected = _getExpectedResult(action↑, data↑);
    assertEq(result, expected);
}
```

Highly likely to be self-referential

Hard not to duplicate code of target

Separate language is better

Multicall

Lack of relation between function being called and multi call means that testing multicall is very hard and ineffective in most cases

-> Centrifuge -> Shortcut clamping. Very poor results.

-> As soon as they changed their code to not require multicall, they quickly gained much higher coverage.

Death of coverage

```
/// @inheritdoc IHub
function execute(PoolId poolId, bytes[] calldata data) external payable {
    require(unlockedPoolId.isNull(), IHub.PoolAlreadyUnlocked());
    require(hubRegistry.isAdmin(poolId, msg.sender), IHub.NotAuthorizedAdmin());

    accounting.unlock(poolId);
    unlockedPoolId = poolId;

    multicall(data);

    accounting.lock();
    unlockedPoolId = PoolId.wrap(0);
}
```

```
/// @dev Makes a call directly to the unclamped handler so
function hub_multicall_clamped() public payable {
    this.hub_multicall{value: msg.value}(queuedCalls);

    queuedCalls = new bytes[](0);
}
```

Per UniV4 Talk

Any time we decouple the action from the affecting coverage

We are messing up the rewards for the fuzzer and we'll have a hard time reaching coverage

Uniswap V4 Example

Personally the only rational way to fuzz UniV4 is via fuzzing it as a set of libraries

Some codebases, like UniV4, and Balancer V2 are not really well explorable.

Our take: Tools + AI Assisted Automated Coverage Strategies

-> Decisions

-> What to clamp

-> Action of Clamping it

-> Show our current work (maybe let’s hardcode a UI / Demo for it)

-> Easier to adapt to tool than build a new tool

The ideal solution

Optimized Generic Fuzzer

Opinionated Fuzzer

Fuzzing Lib / Ad Hoc Solutions

The complete lack of UX changes is very odd to me

Esp for Whitebox fuzzing

Unspoken issue

- Foundry
- Echidna
- Medusa
  (It's all ex ToB maintaining all fuzzers due to diverging incentives)

If we stop experimenting with new techniques then there's really no point in having multiple fuzzers

The main reason these exists is tied to the economic incentives behind them more than a need.